

Exercise 1:

Code:

```
#include <avr/io.h>
#include <util/delay.h>
#define LED_PIN PB0

void setup() {
    DDRB |= (1 << LED_PIN);
}

void loop() {
    PORTB |= (1 << LED_PIN);
    _delay_ms(3000);
    PORTB &= ~(1 << LED_PIN);
    _delay_ms(1000);
}

int main(void) {
    setup();
    while (1) {
        loop();
    }
    return 0;
}
```

Observation and Explanation

If you replace the LED (with its current limiting resistor, 100 Ω) with a **5V fan**, the fan will likely **not turn on** or will run **very weakly/slowly**.

Explanation:

1. **Current Draw:** A typical small DC fan (even a 5V one) requires significantly **more current** than a single LED. LEDs usually draw 10 to 20 mA, while small fans can draw hundreds of milliamps (e.g., 100 mA to 500 mA).
2. **Transistor Limitations:** The small NPN BJT shown in the diagram is usually a general-purpose, low-power transistor (like a BC547 or 2N3904), which has a low maximum **Collector Current (I_C)** rating (often <100 mA or <200 mA).
3. **Operation Failure:** If the fan attempts to draw more current than the BJT can safely handle, the BJT will operate outside its safe zone. It will either fail to saturate (turn fully ON), resulting in a high voltage drop across it and low fan

power, or it could be permanently damaged due to **overheating** or **overcurrent**. The fan won't get enough voltage/current to run properly.

Does it work if we just change a resistor?

No, just changing the 100 Ω resistor will not solve the fundamental issue.

Exercise 2:

Code:

```
const int fanPin = 9;
void setup() {
    pinMode(fanPin, OUTPUT);
}

void loop() {
    int initialSpeed = 51;
    int speedIncrement = (255 - initialSpeed) / 5;

    for (int i = 0; i < 5; i++) {
        int currentSpeed = initialSpeed + (i * speedIncrement);
        if (currentSpeed > 255) {
            currentSpeed = 255;
        }

        analogWrite(fanPin, currentSpeed);

        delay(1000);
    }

    analogWrite(fanPin, 0);

    delay(10000);
}
```

B) Control Fan Speed with a Potentiometer

This exercise modifies part A to use a potentiometer to set the fan's speed during the 5-second ON period.

The potentiometer's wiper is connected to an **Analog Input Pin (e.g., A0)**, which reads a voltage from 0V to 5V. This corresponds to an analog value from 0 to 1023. This value must be **mapped** to the PWM range of 0 to 255.

Exercise 3:

```
const int fanPin = 9;

void setup() {
  pinMode(fanPin, OUTPUT);
}

void loop() {
  analogWrite(fanPin, 255);

  delay(6000);

  analogWrite(fanPin, 0);

  delay(10000);
}
```

The requirement is to use the MOSFET circuit to control the speed of the fan, having the fan **on for 6 seconds** and then **off for 10 seconds**. Since the problem doesn't specify a speed other than "control the speed," we will set the fan to **full speed** (PWM value 255) during the ON period for simplicity.

Exercise 4:

How the Circuit Works and Role of the Opto-coupler

The circuit uses the microcontroller (Arduino/ATmega) to control a low-current LED inside the Opto-coupler

- 1- Operation: When the Arduino output is HIGH, the internal LED turns ON, which illuminates and switches ON the internal phototransistor. This allows the external 5V-7V DC fan to run for the programmed 3 seconds. When the Arduino output is LOW, the fan is OFF for 10 seconds.
- 2- Role of the Opto-coupler: Its primary role is to provide electrical isolation (galvanic isolation). By using light to transmit the signal, it protects the low-voltage Arduino from voltage spikes or noise on the fan's external power circuit.
- using the circuit for 230V load
- NO, the circuit isn't suitable for 230V load
- Explanation:

The internal phototransistor of a standard opto-coupler has low voltage and current ratings. Connecting 230V mains power would exceed its maximum ratings and cause the component to fail⁹. To switch a 230V AC load using an isolation principle, a high-voltage device like a Solid-State Relay (SSR) or a Triac must be used, which is triggered by a Triac Driver Optocoupler